UniGuide

Deployment Plan

UMHackathon 2026, Final Round, Domain 1: AI Systems & Agentic Workflow Automation

Team: Breaking Bank, Team 108, Universiti Malaya

Live deploy: uniguide-blush.vercel.app

Repository: github.com/ZenBen5173/uniguide

Document version: Final Round v2.1, 2 May 2026 (sponsor logos in header)

Table of Contents

1. Executive Summary 3
2. Architecture 3
2.1 Application Layer (Vercel, Singapore) 3
2.2 Data Layer (Supabase, Seoul) 3
2.3 AI Service Layer (Z.AI GLM, with ILMU routable) 4
3. Tech Stack & Why We Chose It 4
4. CI / CD Pipeline 5
4.1 Trunk-Based Deployment 5
4.2 Preview Environments 5
4.3 Pre-Deploy Local Gates 5
5. Secrets Management 5
6. Database Schema Management 6
7. Monitoring & Observability 6
8. Resilience Design 6
9. Backup & Disaster Recovery 7
10. Scalability Plan 7
11. Security & Compliance 8
12. Dependency Hygiene 8
13. Operational Runbook 8
13.1 Common Operations 8
13.2 Top Three Incident Playbooks 9
14. Deployment Roadmap 9
15. Terms & Definitions 9

1. Executive Summary

UniGuide is deployed today on a production-grade stack that places computers as close to UM as possible: the application runs on Vercel's Singapore region (sin1), the database and authentication run on Supabase's Seoul region (ap-northeast-2), and the AI calls go to Z.AI's GLM (the mandatory model under UMHackathon 2026 judging rules). The round-trip time between Vercel and Supabase is around 70 milliseconds, which keeps the user-facing experience snappy.

Deployment is automated. Every push to the main branch on GitHub triggers a Vercel build and deploy in roughly 90 seconds. Database schema changes live as numbered SQL files in the repository (17 of them at the time of writing) and are applied to production through the Supabase tooling. Sensitive values (AI keys, database credentials) live only in Vercel's encrypted environment variables, never in the repository.

The system is designed to fail gracefully. If the AI provider is unreachable, the application falls back to recorded sample responses so the demo never collapses. If the briefing generation step fails after a student submits, a placeholder briefing is written and the application still moves to the coordinator's queue. If the letter-fill step fails, the raw template is written as the letter so the student receives something rather than nothing. None of these fallbacks rely on the AI; they rely on the application code's own error handling.

This document describes the production deployment, the day-two operational practices around it (monitoring, secret rotation, backups), and the staged scaling plan from current capacity to a multi-institution rollout. A glossary of recurring terms is provided at the end (Section 15).

2. Architecture

Three layers, each independently scalable.

2.1 Application Layer (Vercel, Singapore)

- Next.js 15 with the App Router pattern. Roughly 30 server-side API routes.
- Pinned to Vercel's sin1 region (Singapore) via vercel.json; closest available data centre to Malaysia.
- React on the front-end. Tailwind CSS for styling. No CSS-in-JS at runtime, so the page weight stays small.
- Build hygiene: TypeScript strict mode runs on every push; if any type error exists the build fails.

2.2 Data Layer (Supabase, Seoul)

- Postgres 17 with the pgvector extension installed (held ready for semantic SOP search at the next scale step).
- 17 versioned schema migrations, all idempotent (safe to re-run on a partially-applied database).
- Row-Level Security (RLS) on every table that holds personal data: students see only their own records; coordinators (staff/admin role) see all records; AI inserts go through the server-side service-role client which bypasses RLS by design.
- Realtime publication on four tables (applications, application_steps, application_letters, application_messages) with REPLICA IDENTITY FULL set so filtered UPDATE events fire correctly.
- Storage: a private bucket named application-files holds student uploads; access is by signed URL and is scoped to the file's owner or staff.

- Authentication: Supabase one-time-password by email for staff and students. Three demo accounts use static seeded credentials documented openly for hackathon judging.

2.3 AI Service Layer (Z.AI GLM, with ILMU routable)

- Eight AI call sites in lib/glm/: nextStep, generateBriefing, parseDocument, fillLetter, estimateProgress, extractIntent, coassist, studentChat.
- Single client wrapper (lib/glm/client.ts) using the OpenAI npm SDK with Z.AI's OpenAI-compatible base URL. ILMU's client mirrors the same shape so the briefing call swaps between providers behind one environment flag.
- Versioned system prompts in lib/glm/prompts/*.md. The hash of the prompt is recorded on every AI call so traces are reproducible across prompt revisions.
- Mock-mode auto-fallback: every call has a recorded JSON fixture. When the AI key is missing or a real call errors and a fixture is named, the fixture is returned. The user-facing flow never breaks.

3. Tech Stack & Why We Chose It

Layer	Choice	Why this and not the alternative
Framework	Next.js 15 (App Router) plus TypeScript strict plus Zod runtime validation	Server-rendered, edge-friendly, single codebase for student, coordinator, and admin surfaces. TypeScript strict catches contract drift at build; Zod catches it at request time.
Hosting	Vercel sin1, auto-deploy on git push to main	Zero-configuration deploy, preview environments per pull request, region pinned for Malaysia latency. Free tier covers hackathon scale; Pro tier covers pilot scale.
Database	Supabase (Postgres 17 plus Auth, Storage, Realtime, pgvector)	One managed service for the whole data plane. RLS-first design. Realtime publication built in (no custom WebSocket service to operate). pgvector ready for semantic SOP search at the next scale step.
AI provider	Z.AI GLM-4.6 plus GLM-4.5-flash; ILMU GLM-5.1 routable for briefing	Z.AI is the mandatory model per UMHackathon 2026 rules. ILMU is the Malaysian-sovereign LLM by YTL AI Labs and UM, used for the Bahasa-heavy coordinator

Layer	Choice	Why this and not the alternative
		briefing call only, behind an environment flag.
PDF processing	pdf-parse for text extraction; native browser print for letter rendering	Avoids heavy server-side PDF generators in the MVP. pdf-parse handles real-text PDFs well; image-only PDFs are explicitly rejected (OCR is on the roadmap).
Testing	Vitest with one resilience smoke test	Confidence at hackathon scale comes primarily from manual demo runs and the fully-typed schema. Integration tests are scheduled for the post-finals ramp.

4. CI / CD Pipeline

4.1 Trunk-Based Deployment

- The main branch is deployable. Vercel watches the GitHub repository and rebuilds on every push.
- Build steps run by Vercel: npm install, then next build. The build fails if TypeScript fails (tsc --noEmit) or if any imported module fails to resolve.
- Average build time: about 90 seconds. The cold path is GitHub webhook, then Vercel build, then Vercel deploy, then atomic alias swap on the production URL.

4.2 Preview Environments

- Every pull request automatically receives a preview deployment URL with its own ephemeral state.
- Preview environments share the production Supabase by default (acceptable at hackathon scale). Moving previews to a staging Supabase project is a one-line vercel.json change for the pilot phase.

4.3 Pre-Deploy Local Gates

Three local commands, run before every push, taking under 30 seconds combined:

- npm run typecheck (TypeScript strict, must be clean)
- npm run lint (Next.js lint, must be clean)
- npm test:run (Vitest, the resilience smoke test must pass)

These are policy today rather than enforced by CI. CI gating (running the same three commands automatically on every pull request) is the next item on the post-finals deployment roadmap.

5. Secrets Management

- The .env file is gitignored. The .env.example file lists every variable name with a brief description but no values.
- Production secrets live exclusively in Vercel Environment Variables, encrypted at rest, scoped per environment (production, preview, development).
- The Supabase service-role key is server-only; it is never exposed to the browser. The NEXT_PUBLIC_ prefix denotes public-safe values only.
- Z.AI API key, ILMU API key, and Supabase service-role key all rotate manually on a quarterly cadence (operator action; no automated rotation in MVP).
- Demo accounts use static seeded credentials documented openly. They are intentional and shipped for judging.

6. Database Schema Management

- Versioned SQL files in supabase/migrations/. Each is idempotent (uses IF NOT EXISTS and conditional drops) so re-running on a partially-applied database is safe.
- Application path: npx supabase db push if the CLI is linked to the project, or paste each file into the Supabase SQL editor in order. Both methods leave the schema in the same state.
- Migrations 0016 (glm_trace_endpoint_widen) and 0017 (ai_chat_and_escalations) are the most recent. They were applied to production at the start of the final-round physical sub-phase to align the live database with the new code.
- RLS policy changes go through the same migration system; they are not applied via the dashboard.
- Rollback strategy: an inverse migration where possible, or restoring from Supabase's daily snapshot otherwise. There is no automated rollback path in the MVP; this is a known acceptable risk for hackathon scale.

7. Monitoring & Observability

- **Application logs.** Vercel native log streams, filterable per route. 7-day retention on Hobby; 30-day on Pro.
- **AI audit log.** The glm_reasoning_trace table records every AI call's endpoint, model, prompt hash, input summary, output, latency, input tokens, output tokens, cache hit, and retry count.
- **Operational dashboards.** /admin/glm-traces (per-call audit), /admin/cost (14-day token spend rolled up by model, endpoint, and day, with a USD estimate), /admin/analytics (KPIs and per-procedure status mix).
- **Database health.** Supabase native dashboard, query latency, connection pool, replication lag, storage usage.
- **Alerting.** Vercel Slack integration set up for build failures. Application-level alerting (for example, SLA-breach detection) is on the post-finals roadmap.

8. Resilience Design

The system is engineered so that no single AI failure leaves an application in a broken state.

- **Mock-mode auto-fallback.** On any AI call error where a mock fixture is named, the recorded fixture is returned and the trace row is marked accordingly. The application advances normally; the lavender Demo banner does NOT appear (live mode is preserved).
- **Briefing fallback on submit.** If the briefing generation step fails, the submit endpoint writes a placeholder briefing flagged for coordinator regeneration AND still flips the application to submitted. The student is never left in a half-submitted state.
- **Letter fallback on decide.** If the letter-fill step fails, the raw template (with placeholders unfilled) is written as the letter. The student receives a letter that needs cleanup; the application doesn't get stuck.
- **Preview-step fallback.** If the preview-step endpoint fails, the modal offers a Send-without-preview path that fires the legacy direct-fire decide. Coordinators are never blocked.
- **Student chat fallback.** If the student chat call fails, the panel inserts a graceful AI reply that itself suggests escalation, so the student has an action.
- **Trace failures don't block.** The trace writer catches and logs; the user-facing flow continues if the audit insert fails.

9. Backup & Disaster Recovery

- Database: Supabase daily snapshots (7-day retention on the free tier; can upgrade to point-in-time recovery on Pro for 7-day rollback).
- Storage: the application-files bucket is replicated within Supabase's managed object store; durability matches the rest of the data plane.
- Code and migrations: GitHub remote (off-Supabase). Rolling back code is a git revert and push; Vercel rebuilds within about 90 seconds.
- Recovery time objective (RTO): under 1 hour for a code rollback; under 4 hours for a database restore.
- Recovery point objective (RPO): up to 24 hours on the free tier (the most recent daily snapshot); under 5 minutes on Pro point-in-time recovery.

10. Scalability Plan

Concern	Current capacity	Scale plan
Concurrent users	Vercel and Supabase scale horizontally; comfortable to about 5,000 concurrent at hackathon scale.	Pro tier upgrades plus Postgres connection pooler (Supavisor) when concurrent reaches 10,000 or more.
Coordinator inbox query	Loads all active applications in one query; fine to about 1,000 active.	Add cursor pagination plus a status-mix index. Scheduled for the post-finals ramp.
SOP retrieval	Fetches all chunks per procedure (works for 5 procedures with about 50 chunks total).	Activate pgvector semantic search in lib/kb/retrieve.ts when the catalogue exceeds about 30 procedures.

Concern	Current capacity	Scale plan
AI rate limits	Z.AI's per-minute limits comfortably cover hackathon-scale demos.	Pre-emptive request shaping at the application layer; queue plus retry-with-backoff in client.ts when sustained traffic crosses thresholds.
Storage	About 1 MB per application on average × 50 applications = 50 MB on the free tier.	Pro tier raises bucket size to 100 GB; single-tenant; fine through Year 2.

11. Security & Compliance

- RLS on every table holding personal data. Policies enforce ownership and role separation.
- Service-role secrets are never exposed to the browser; all operations that bypass RLS go through server-side route handlers.
- Storage signed URLs expire in minutes; there are no public links to student documents.
- Common web attack posture: Zod validates every request body shape; SQL injection is structurally prevented by Supabase's PostgREST and RPC layer; cross-site-scripting (XSS) surface is minimised by React's default escaping.
- PDPA (Personal Data Protection Act 2010, Malaysia) alignment: data is hosted in ap-northeast-2 (Seoul) under a regional data-processing agreement; a per-institution Data Processing Agreement is signed before pilot deployment; access is logged via Supabase's audit trail; right-to-erasure is supported via account deletion plus cascading foreign-key deletes.
- Hackathon T&Cs: per UMHackathon 2026 §9, all rights to the code submission belong to the domain collaborators.

12. Dependency Hygiene

The repository has 7 moderate-severity npm audit warnings at the time of writing. Every one of them is in transitive development dependencies (esbuild via vite via vitest, and postcss via Next 15). The npm audit fix --force resolution would force-downgrade Next.js from version 15 to version 9, which is a major architectural regression. We have explicitly chosen NOT to apply --force. The vulnerabilities have no exposure in production code paths (vite and vitest are development-only; postcss runs only at build time).

The post-finals dependency-hygiene plan is to upgrade vitest to a version whose esbuild transitive dependency is patched without breaking changes. That work is queued for Q3 2026.

13. Operational Runbook

13.1 Common Operations

- Add a new procedure: insert into the procedures table; drop the SOP markdown into lib/kb/seed/; run npm run seed:kb; add letter templates via /admin. About 15 minutes per procedure.

- Update an SOP: paste, URL, or PDF upload via /admin/procedures/[id], then click Upload SOP. The change applies immediately for new applications and any in-flight applications that have not passed the affected step yet.
- Reset demo data: any demo-tile sign-in fires /api/demo/reset automatically; or manually from /admin → Demo reset.
- Apply a schema migration: paste the SQL into the Supabase SQL editor, or run npx supabase db push if the CLI is linked.
- Rotate the Z.AI API key: Vercel Project Settings → Environment Variables → ZAI_API_KEY → set new value → trigger redeploy. The old key remains valid until you revoke it on the Z.AI side.

13.2 Top Three Incident Playbooks

- **Z.AI provider down.** Set GLM_MOCK_MODE=true on Vercel envs to force fallback fixtures; trigger a redeploy. The live demo continues against recorded responses with the lavender banner shown.
- **Bad migration breaks production.** Revert the schema change via inverse migration; if not possible, restore the most recent Supabase snapshot. A code rollback (git revert) restores the prior schema expectation if needed.
- **Letter sent with bad fact (hallucination check missed).** The 5-minute undo lets the coordinator retract within the window. Past the window, manually flag the decision in application_decisions and have the coordinator re-issue the letter.

14. Deployment Roadmap

- **Q3 2026, operational hardening:** CI gating (typecheck plus lint plus tests on every PR); rate limiting on public APIs (Upstash Redis is already in the stack); separate staging Supabase; SLA-breach Slack alerts.
- **Q4 2026, pilot readiness:** real email delivery via Resend or Postmark; OCR for scanned PDFs; per-faculty coordinator scoping; tamper-proof append-only audit log for decisions.
- **Q1 2027, multi-institution:** multi-tenant isolation; per-institution branding; staging environment for SOP testing; data residency options.
- **Q2 2027, regional:** PWA shell for mobile; ASEAN regional Data Processing Agreements; localisation framework for non-EN/MS languages.

15. Terms & Definitions

Recurring jargon used throughout this document, defined in plain language.

Term	Plain-language definition
AI hallucination	When the AI states something untrue or invents a fact (for example, the wrong CGPA in a letter). UniGuide's hallucination check uses a two-layer approach: a regex first layer that catches factual mismatches (for example, CGPA or name discrepancies), followed by a Z.AI GLM second layer acting as an LLM-as-a-judge to catch subtler errors before any letter is sent.

Term	Plain-language definition
CI / CD	Continuous Integration and Continuous Deployment; the automated pipeline that builds, tests, and deploys the application on every push to main.
Edge function	A small piece of code that runs at a content delivery network's edge nodes, close to the user, instead of in a single central data centre. Reduces latency.
FTE	Full-Time Equivalent; one full-time staff member working a full week. '0.2 FTE' means 20% of one full-time staff member.
GLM	Z.AI's family of large language models (GLM-4.6 and GLM-4.5-flash). The mandatory model for UMHackathon 2026.
Idempotent migration	A schema change that produces the same end state whether it is run once or many times. UniGuide's migrations all use IF NOT EXISTS and conditional drops so they're safe to re-run.
ILMU	Malaysia's sovereign large language model, developed by YTL AI Labs and Universiti Malaya. Routable as the briefing provider in UniGuide for Bahasa-heavy content.
Mock fallback	Recorded sample AI responses (in tests/fixtures/glm/) that the application returns when the real AI provider is unavailable, so the demo never collapses.
PDPA	Personal Data Protection Act 2010 (Malaysia); the law governing how personal data is collected, processed, and stored.
Pgvector	A Postgres extension that stores and searches vector embeddings, used for semantic similarity search. Installed but not yet active in UniGuide; ready for the next scale step.
RLS	Row-Level Security; a Postgres feature that restricts which rows each user can read or write. UniGuide enforces RLS on every table holding personal data.
RPO / RTO	Recovery Point Objective (how much data you can afford to lose, measured in time) and Recovery Time Objective (how long it takes to restore service after a failure).
Service-role key	A privileged Supabase API key that bypasses RLS. Used only by server-side code; never exposed to the browser.
SLA	Service-Level Agreement; the response-time commitment per submitted case (for example, 'coordinators respond within 48 hours of submit').

Term	Plain-language definition
SOP	Standard Operating Procedure; the official UM document describing how a procedure (scholarship, FYP, deferment, etc.) is meant to run. UniGuide's AI reads the SOP at every step.
Supavisor	Supabase's connection pooler for Postgres, used to handle high concurrent connection counts at scale.
Trunk-based deployment	A workflow where the main branch is always deployable; every push to main triggers a production deploy. Contrasts with long-lived release branches.

End of Deployment Plan